

Rapport Projet Annuel Kojin

ESGI — Mastère Expert IA & Big Data 5IABD — Classe A Alternance RO

PROJET ANNUEL

Kōjin

Composition automatisée de bentos nutritionnellement optimisés à partir de la base Open Food Facts

Rapport de projet

Équipe projet MOREL Tom EL KHEIR Douaa ELKADOURI Hajar ZAIDI Yassine

Année académique 2025 – 2026

Application déployée — accès public : <http://kojin-alb-1461380392.us-east-1.elb.amazonaws.com>

Sommaire

1. Introduction et contexte
2. Présentation du projet 2.1 Nom et descriptif 2.2 Pour qui, pour quoi faire 2.3 Matières IABD mobilisées
3. Veille technologique et existant
4. Spécifications fonctionnelles 4.1 Utilisateurs types 4.2 Description des cas d'usage 4.3 Maquette / interface
5. Spécifications techniques et architecture 5.1 Vue d'ensemble de la stack 5.2 Calcul des objectifs nutritionnels 5.3 Algorithme de composition des bentos 5.4 Pipeline d'acquisition et de préparation des données 5.5 Recommandation par renforcement (RL Planner) et génération de contenu (LLM)
6. Architecture globale
7. Exigences transverses
8. Déploiement 8.1 Réseau et sécurité (VPC, security groups) 8.2 Conteneurisation et dimensionnement de la tâche ECS Fargate 8.3 Exposition via l'Application Load Balancer et accès public 8.4 Stockage des données (S3 et EFS) 8.5 Identités et permissions (IAM) 8.6 Pipeline de mise en production 8.7 Supervision, alertes et FinOps 8.8 Estimation des coûts (FinOps) 8.9 Checklist de mise en production et nettoyage 8.10 Variables d'environnement principales 8.11 Incidents de déploiement rencontrés et résolus
9. Organisation du projet et travail en équipe

10. Qualité d'usage, tests et validation
 11. Gestion des risques
 12. Bilan de réalisation
 13. Glossaire
 14. Annexes
 15. Conclusion
-

1. Introduction et contexte

Ce document constitue le rapport du projet annuel réalisé dans le cadre de la 5ème année du Mastère Expert IA & Big Data de l'ESGI, filière 5IABD (classe A, alternance RO). Il présente le projet Kōjin, une application web qui compose automatiquement des bentos nutritionnellement optimisés à partir de la base de données ouverte Open Food Facts, en fonction du profil, de l'activité physique et des objectifs de l'utilisateur, et qui propose en complément une planification interactive par apprentissage par renforcement ainsi qu'une interface d'exploration du catalogue en langage naturel.

Le projet a été mené en équipe de quatre étudiants, suivant le déroulé pédagogique en séances de suivi (proposition de projet, spécifications fonctionnelles et techniques, prototype, bilan avant soutenance) avant la soutenance finale. Ce rapport reprend cette progression : contexte et motivation, spécifications fonctionnelles et techniques, choix d'architecture, exigences transverses (automatisation, cloud, sécurité, FinOps), déploiement effectivement réalisé sur AWS, organisation du travail en équipe, puis bilan et perspectives. L'application est accessible publiquement à l'adresse <http://kojin-alb-1461380392.us-east-1.elb.amazonaws.com>.

2. Présentation du projet

2.1 Nom et descriptif

Kōjin est une application web qui calcule les besoins nutritionnels journaliers d'un utilisateur puis résout, sous contraintes, la composition de un à cinq bentos équilibrés en s'appuyant sur le catalogue de produits Open Food Facts, tout en respectant un régime alimentaire donné (vegan, végétarien, halal, casher, sans gluten, bio). L'application va au-delà de ce moteur d'optimisation initial et propose quatre espaces fonctionnels accessibles depuis une navigation unique : un profil utilisateur persistant, le planificateur de bentos par optimisation (Bento Planner), une page d'exploration du catalogue en langage naturel, et un planificateur de journée par politique d'apprentissage par renforcement (RL Planner).

2.2 Pour qui, pour quoi faire

Le projet s'adresse à toute personne souhaitant structurer son alimentation autour d'objectifs physiques précis (sèche musculaire, recomposition corporelle, prise de masse) sans avoir à calculer manuellement les portions de chaque aliment. Il vise en particulier les pratiquants de sport en salle, les personnes suivant un rééquilibrage alimentaire, et plus largement toute personne curieuse de voir

ce qu’une approche data / optimisation peut apporter à un sujet aussi concret que la préparation de repas.

Au-delà de la note, ce projet a été pensé comme une carte de visite technique : il combine des compétences directement mobilisables en entreprise — ingestion et nettoyage de données à grande échelle (Polars, Parquet), modélisation mathématique (optimisation sous contraintes), apprentissage par renforcement (NCF + PPO), intégration cloud (AWS Bedrock, S3, EFS, ECS Fargate) et restitution utilisateur (Streamlit) — ce qui correspond aux profils Data Engineer / Data Scientist visés par les membres de l’équipe à l’issue de leur alternance.

2.3 Matières IABD mobilisées

Matière IABD	Mise en œuvre concrète dans le projet
Big Data & traitement à l’échelle	Ingestion et filtrage du dataset Open Food Facts (6,7 Go, format Parquet) avec Polars, traitement par groupes de lignes
Intelligence artificielle / optimisation	Modélisation du problème de composition de bento comme un système linéaire sous contraintes (NNLS puis BVLS)
Intelligence artificielle / apprentissage par renforcement	Politique PPO résiduelle entraînée sur un simulateur de préférences NCF (Neural Collaborative Filtering) construit sur les avis réels Food.com, pour la planification interactive d’une journée de repas
IA générative / LLM	Appel à un modèle de langage managé (Amazon Bedrock) pour la traduction de questions en langage naturel vers des requêtes SQL DuckDB (page Exploration)
Cloud & DevOps	Déploiement conteneurisé sur AWS (ECS Fargate en architecture ARM64/Graviton, S3, EFS, Application Load Balancer), gestion des accès via rôle IAM plutôt que clé statique
FinOps	Choix raisonné du modèle Bedrock au regard du coût et de la disponibilité immédiate, budget AWS avec alertes automatiques, optimisation de la taille de l’image Docker
Développement d’application	Conception d’une interface Streamlit multipage et de la logique métier associée en Python

3. Veille technologique et existant

Le marché compte plusieurs catégories d’applications proches : les applications de suivi nutritionnel (type Yazio, MyFitnessPal) qui laissent l’utilisateur choisir et journaliser ses aliments a posteriori, les services de « meal prep » sur abonnement qui livrent des plats déjà composés sans marge de

personnalisation fine, et des calculateurs de macros qui s’arrêtent au calcul des besoins sans proposer de composition concrète d’assiette.

Solution	Logique	Catalogue produits	Composition automatique	Coût
Yazio / MyFitnessPal	Journalisation a posteriori	Fermé, contributions utilisateurs	Non — l’utilisateur choisit puis vérifie	Freemium
Meal prep sur abonnement	Menus imposés par le prestataire	Fermé, recettes propriétaires	Oui, mais non personnalisable finement	Abonnement payant
Calculateurs de macros seuls	Calcul des besoins uniquement	Aucun	Non	Gratuit
Kōjin	Part des objectifs, résout la composition	Ouvert (Open Food Facts, licence ODbL)	Oui, par optimisation sous contraintes ou par politique RL	Gratuit / coût cloud marginal

La différence de Kōjin tient donc à l’inversion de la logique habituelle : plutôt que de laisser l’utilisateur composer son repas puis vérifier s’il correspond à ses objectifs, l’application part des objectifs et résout un problème d’optimisation (ou interroge une politique apprise) pour proposer directement une composition chiffrée en grammes, à partir d’un catalogue ouvert.

Cette veille a permis d’identifier les limites à anticiper : la qualité hétérogène du renseignement nutritionnel sur Open Food Facts, ce qui a motivé le filtrage sur les produits commercialisés en France et non obsolètes ainsi que l’exclusion des produits ultra-transformés (NOVA 4). Une veille complémentaire sur les solveurs d’optimisation a confirmé le choix d’une approche NNLS puis BVLS plutôt qu’un solveur de programmation linéaire en nombres entiers. Sur le volet recommandation interactive, la veille s’est appuyée sur les travaux de Liu et al. (2024), “*An Interactive Food Recommendation System Using Reinforcement Learning*”, qui ont directement inspiré l’architecture du RL Planner.

4. Spécifications fonctionnelles

4.1 Utilisateurs types

- Un pratiquant de musculation en phase de sèche ou de prise de masse, cherchant une cible calorique et protéique précise répartie sur la journée
- Une personne suivant un régime alimentaire contraint (vegan, halal, casher, sans gluten, bio) qui souhaite un filtrage fiable des produits
- Un utilisateur occasionnel qui veut simplement une idée de repas équilibré sans réflexion nutritionnelle poussée, éventuellement en laissant une politique apprise proposer directement une journée complète

Persona	Profil	Besoin principal	Fonctionnalité clé mobilisée
Le sportif en sèche	25-35 ans, 4 séances de sport/semaine, objectif de perte de masse grasse	Un contrôle strict des calories et un apport protéique élevé sans calcul manuel	Objectif « Sèche musculaire », Bento Planner, bento désigné pour la protéine animale
La personne à régime contraint	Vegan ou pratiquant un régime religieux	Un filtrage fiable, sans devoir vérifier chaque étiquette	Filtres de régime (tags booléens) appliqués avant optimisation
L'utilisateur occasionnel	Curieux de l'outil, pas d'objectif sportif précis	Une idée de repas équilibré rapide, sans configuration complexe	RL Planner — génération immédiate d'une journée complète par la politique apprise

4.2 Description des cas d'usage

L'application se compose de quatre pages accessibles depuis une navigation unique (`st.navigation`) : **Profil**, **Bento Planner**, **Exploration des ingrédients** et **RL Planner**.

Scénario 1 — Renseignement du profil. L'utilisateur renseigne son profil (genre, âge, poids, taille), son niveau d'activité quotidienne, sa fréquence sportive, son objectif physique et un éventuel régime alimentaire, ainsi que son plan de repas (nombre de repas, catégories, proportions caloriques). Ces informations sont persistées en base SQLite (`kojin.db`), et non plus seulement le temps de la session, ce qui permet de retrouver son profil d'une visite à l'autre via un pseudo.

Scénario 2 — Consultation des objectifs journaliers. L'application affiche les objectifs nutritionnels journaliers calculés (énergie, protéines, lipides, glucides, portion de légumes) selon la formule de Mifflin–St Jeor.

Scénario 3 — Configuration et composition des bentos (Bento Planner). L'utilisateur choisit le nombre de bentos (un à cinq), ajuste la fraction calorique de chacun via des curseurs couplés, désigne le bento recevant la protéine animale, puis compose : le solveur hybride NNLS + BVLS affiche pour chaque bento les aliments retenus, leurs quantités et leur contribution macro-nutritionnelle.

Scénario 4 — Exploration en langage naturel. L'utilisateur pose une question en langage naturel sur le catalogue produits (ex. « Quels produits vegan ont plus de 25 g de protéines pour 100 g ? ») ; un modèle de langage (Amazon Bedrock) génère une requête SQL exécutée en lecture seule sur une base DuckDB matérialisée à partir du catalogue.

Scénario 5 — Planification interactive par renforcement (RL Planner). L'utilisateur se connecte par pseudo ; une politique PPO entraînée hors ligne sur les préférences réelles d'utilisateurs Food.com génère simultanément une recette pour chaque créneau de repas défini dans le profil. L'utilisateur peut demander une nouvelle proposition pour un créneau donné (« Changer ») ou valider la journée entière, ce qui déclenche la résolution parallèle de chaque ingrédient abstrait en

produit réel Open Food Facts (via un IngredientResolver LLM→SQL→DuckDB) et affine l’embedding de préférence de l’utilisateur.

Scénario 6 — Absence de données locales (cas limite). Si le catalogue préparé n’est pas présent au démarrage (premier lancement local), un bouton dédié permet de déclencher le pipeline de préparation directement depuis l’interface.

4.3 Maquette / interface

L’interface repose sur Streamlit avec une barre latérale de connexion/navigation et quatre pages en zone principale. La direction artistique s’inspire de la typographie japonaise (police serif) pour donner à l’outil une identité visuelle sobre et cohérente avec son nom, avec un contraste clair/sombre soigné entre la barre latérale (fond sombre, texte clair) et le contenu principal (fond clair, texte sombre).

5. Spécifications techniques et architecture

5.1 Vue d’ensemble de la stack

Composant	Rôle
Streamlit	Interface web multipage et interactions utilisateur
Polars, DuckDB	Chargement/filtrage du catalogue produits et requêtage SQL en lecture seule (page Exploration)
SciPy (nnls, lsq_linear/BVLS)	Solveur d’optimisation linéaire sous contraintes pour la composition des bentos
PyTorch (NCF + PPO résiduel)	Politique de recommandation interactive du RL Planner (Liu et al., 2024)
LangChain (core, aws, anthropic, openai)	Abstraction d’appel aux modèles de langage, NL→SQL
Hugging Face Hub	Téléchargement du dataset Open Food Facts (food.parquet, ~6,7 Go)
Kaggle / kagglehub	Téléchargement du dataset Food.com (recettes et avis) pour l’entraînement du RL Planner
boto3 / Amazon Bedrock	Authentification par rôle IAM, appel du modèle de langage en production
AWS ECS Fargate (ARM64), S3, EFS, ALB	Hébergement, stockage et exposition de l’application en production

5.2 Calcul des objectifs nutritionnels

Le métabolisme de base (BMR) est calculé avec la formule de Mifflin–St Jeor, puis ajusté par un multiplicateur combinant l’activité quotidienne et la fréquence sportive (plafonné à 1,95) pour obtenir la dépense énergétique totale (TDEE). Les cibles finales sont dérivées de ce TDEE : l’énergie

journalière est modulée par un facteur propre à l'objectif (0,90 en sèche, 1,00 en maintien, 1,15 en prise de masse), les protéines sont calculées au poids de corps (1,6 à 2,2 g/kg selon l'objectif), les lipides sont dérivés d'un pourcentage de l'énergie totale, et les glucides comblent le reste de l'énergie journalière avec un plancher de 50 g.

Exemple chiffré — homme, 28 ans, 78 kg, 1,80 m, activité modérée, 4 séances de sport/semaine, objectif « sèche musculaire » :

Étape	Calcul	Résultat
BMR (Mifflin–St Jeor)	$10 \times 78 + 6,25 \times 180 - 5 \times 28 + 5$	$\approx 1\,744$ kcal
Multiplicateur d'activité	activité modérée + fréquence sportive (plafonné à 1,95)	$\approx 1,55$
TDEE	$1\,744 \times 1,55$	$\approx 2\,703$ kcal
Énergie cible (sèche, facteur 0,90)	$2\,703 \times 0,90$	$\approx 2\,433$ kcal
Protéines (2,2 g/kg en sèche)	$2,2 \times 78$	≈ 172 g
Lipides (25 % de l'énergie, /9)	$(2\,433 \times 0,25) / 9$	≈ 68 g
Glucides (reste de l'énergie, /4)	$(2\,433 - 4 \times 172 - 9 \times 68) / 4$	≈ 264 g

5.3 Algorithme de composition des bentos

Pour chaque bento, le problème est modélisé comme un système linéaire sous contraintes : un vecteur cible (calories, protéines, lipides, glucides, fibres, portion de légumes) doit être approché par une combinaison de quantités d'aliments, bornée par une portion maximale par défaut de 200 g.

La résolution est hybride en deux temps : un premier passage par NNLS présélectionne les produits pertinents parmi le catalogue filtré, puis un second passage par BVLS (`lsq_linear, method="bvls"`) affine les quantités sur ce sous-ensemble restreint. Un post-traitement métier applique des règles de bon sens : au maximum une source de protéine animale par jour et uniquement sur le bento désigné, au maximum une huile ajoutée, et aucun doublon d'aliment d'un bento à l'autre.

En notant x le vecteur des quantités des n produits présélectionnés, M la matrice $n \times k$ des teneurs en nutriments et c le vecteur cible du bento, le second passage s'écrit : minimiser $\|M^T x - c\|^2$ sous contrainte $0 \leq x \leq \text{portion_max}$.

5.4 Pipeline d'acquisition et de préparation des données

Le script `data_prep_nutriments.py` télécharge le dataset Open Food Facts (`openfoodfacts/product-database`, ~6,7 Go au format Parquet, via Hugging Face Hub et le protocole Xet) puis le traite en streaming par groupes de lignes. Le traitement filtre les produits commercialisés en France et non obsolètes, extrait cinq macro-nutriments, ajoute des indicateurs booléens de régime (vegan, halal, végétarien, sans gluten, casher, sans huile de palme, bio) et de catégorie (viande, poisson, laitier, pain), et exclut les produits ultra-transformés (NOVA 4). Le résultat, un CSV d'environ 95 Mo (145

021 produits retenus lors de l'exécution de référence), est directement exploitable par l'application ou déposé sur S3 pour la production.

5.5 Recommandation par renforcement (RL Planner) et génération de contenu (LLM)

RL Planner. L'implémentation suit Liu et al. (2024). Une phase d'entraînement hors ligne unique construit, à partir du dataset Food.com (recettes et avis réels, filtré à 50 000 recettes et 2 855 utilisateurs actifs), un simulateur de préférences par Neural Collaborative Filtering (NCF), puis entraîne une politique PPO résiduelle à maximiser les notes prédites par ce simulateur. Le checkpoint produit (~529 Mo) embarque la matrice de préférence NCF, l'état fusionné (FusedState) et les poids de la politique. En inférence, un nouvel utilisateur Kōjin (absent de l'entraînement) démarre sur un état moyen ("goût moyen" Food.com) qui se personnalise par accumulation des repas validés au fil des sessions, via un embedding utilisateur affiné par 30 pas de descente de gradient après chaque validation de journée.

Exploration en langage naturel. La page Exploration traduit une question en langage naturel en requête SQL DuckDB via un modèle de langage appelé par LangChain. En production, ce modèle est invoqué via Amazon Bedrock, authentifié par le rôle IAM attaché à la tâche ECS — sans clé d'API stockée dans le code ni dans l'image. Le choix du modèle de référence a évolué au cours du projet (voir §8.11) : Amazon Nova Pro (us.amazon.nova-pro-v1:0) est le modèle finalement retenu en production, un modèle Anthropic Claude ayant été testé en premier lieu mais nécessitant une validation administrative supplémentaire (formulaire d'usage prévu, spécifique aux modèles tiers sur Bedrock) non compatible avec le calendrier du projet.

6. Architecture globale



Le flux applicatif se lit ainsi : l'utilisateur adresse ses requêtes HTTP à l'Application Load Balancer, public et accessible depuis n'importe quel poste connecté à Internet, qui les répartit vers l'unique tâche ECS Fargate exécutant le conteneur Streamlit. Cette tâche monte un volume EFS sur `/app/data` : contrairement à une tâche Fargate « nue » dont le disque est perdu à chaque redémarrage, ce volume conserve les profils utilisateurs (`kojin.db`) et le checkpoint du RL Planner (`reciperl.pt`, trop volumineux pour être embarqué dans l'image Docker) d'un déploiement à l'autre. La tâche invoque Amazon Bedrock à la demande, en s'authentifiant via le rôle IAM qui lui est attaché plutôt que par une clé statique.

7. Exigences transverses

- **Automatisation** : aucune étape manuelle de transfert de données en production ; la préparation du catalogue peut être redéclenchée depuis l'interface elle-même si le CSV est absent
- **Évolutivité** : le pipeline de préparation est rejouable pour intégrer les mises à jour du dataset Open Food Facts ; le volume EFS découple la persistance des données du cycle de vie de la tâche de calcul
- **Data et traitement dans le cloud** : hébergement du CSV et des artefacts persistants sur S3/EFS, calcul et service applicatif sur ECS Fargate, appel de modèle via Bedrock
- **Disponibilité, sécurité, légalité** : exposition via un Application Load Balancer public, accès au modèle par rôle IAM plutôt que par clé statique, données sources sous licence ouverte ODbL
- **FinOps** : dimensionnement de la tâche ajusté au réel besoin (2 vCPU / 8 Go pour supporter le RL Planner), architecture ARM64/Graviton moins coûteuse que x86, image Docker optimisée (torch CPU-only), budget AWS avec alerte automatique par e-mail à 50 % de la dépense réelle et 100 % de la dépense prévisionnelle

8. Déploiement

L'application est packagée pour un déploiement conteneurisé sur AWS, orchestré par ECS Fargate derrière un Application Load Balancer, avec les données produits et les artefacts persistants stockés respectivement sur S3 et EFS. Le déploiement décrit ci-dessous est celui **effectivement réalisé** (procédure pas à pas dans `DEPLOYMENT.md` du dépôt), et diffère sur plusieurs points d'un déploiement générique de référence — ces écarts, et les incidents rencontrés en les découvrant, sont documentés en §8.11.

8.1 Réseau et sécurité (VPC, security groups)

Le déploiement s'appuie sur les sous-réseaux publics du VPC par défaut de la région **us-east-1** et repose sur trois groupes de sécurité distincts, appliquant le principe de moindre privilège réseau :

- **kojin-alb-sg** : autorise le trafic entrant sur les ports 80 (et 443 en prévision d'un futur certificat TLS) depuis Internet (`0.0.0.0/0`)

- **kojin-svc-sg** : n'autorise le port applicatif 8501 (Streamlit) qu'en provenance du security group de l'ALB — la tâche ECS n'est donc jamais directement exposée sur Internet
- **kojin-efs-sg** : n'autorise le port NFS 2049 qu'en provenance du security group du service ECS

Streamlit reposant sur des WebSockets, le target group active la persistance de session (sticky sessions, cookie de répartiteur, durée 24 h) en plus du contrôle de santé applicatif sur `/_stcore/health`.

8.2 Conteneurisation et dimensionnement de la tâche ECS Fargate

L'application est packagée dans une image Docker (`python:3.12-slim`) contenant le code Python, ses dépendances et les pages `app_pages/`, ainsi que le package `reciperl/` nécessaire au RL Planner. Cette image est poussée vers Amazon ECR (`kojin-streamlit`), puis référencée dans une définition de tâche ECS précisant :

- **CPU / mémoire : 2 vCPU / 8 Go** — dimensionnement relevé par rapport à un déploiement sans RL Planner (qui se contenterait de 1 vCPU / 4 Go), car PyTorch doit charger en mémoire le checkpoint `reciperl.pt` (matrice de préférence $2\,855 \times 39\,280$) en plus de Polars/SciPy et du cache Streamlit du catalogue ;
- **Architecture : ARM64 (Graviton)** — l'image ayant été construite nativement sur un poste de développement Apple Silicon, la tâche est configurée en `runtimePlatform`:
`{cpuArchitecture: ARM64}` plutôt que reconstruite pour `x86_64`, ce qui évite l'émulation et réduit le coût de calcul ;
- **Port exposé** : 8501 ;
- **Volume EFS** monté sur `/app/data` (voir §8.4) ;
- **Contrôle de santé applicatif** sur `/_stcore/health` toutes les 30 secondes.

Le service ECS maintient le nombre de tâches souhaité (1 en régime de démonstration), redémarre automatiquement une tâche défaillante, et peut être relié à une politique d'auto-scaling ciblant 60 % d'utilisation CPU (borne de 1 à 4 tâches) pour absorber les pics de charge.

8.3 Exposition via l'Application Load Balancer et accès public

L'ALB (`kojin-alb`) reçoit le trafic entrant, répartit les requêtes vers le groupe cible constitué de la tâche ECS en cours d'exécution sur le port 8501, et retire automatiquement toute tâche dont le contrôle de santé échoue. L'application est exposée en HTTP sur le port 80 (un listener HTTPS avec certificat ACM et redirection `80→443` est documenté comme évolution de mise en production, cf. §12.2).

L'application est accessible publiquement, depuis n'importe quel poste connecté à Internet (pas seulement la machine de déploiement), à l'adresse :

<http://kojin-alb-1461380392.us-east-1.elb.amazonaws.com>

8.4 Stockage des données (S3 et EFS)

Deux mécanismes de stockage coexistent, avec des rôles distincts :

- **Amazon S3** (`kojin-data-<compte>`, versioning activé, accès public bloqué) héberge le CSV produits et sert de copie de secours du checkpoint RL. Le CSV est téléchargé par la tâche au démarrage via `DATA_S3_URI` s'il n'est pas déjà présent localement.
- **Amazon EFS** (`kojin-data`, monté sur `/app/data` via un access point dédié, chiffrement en transit activé) assure la **persistance** de `kojin.db` (profils utilisateurs, plans de repas, embeddings RL) et de `reciperl.pt` (checkpoint RL, ~529 Mo) à travers les redémarrages et redéploiements de la tâche — une tâche Fargate étant par nature sans état, ces données seraient sinon perdues à chaque mise à jour du service. Le volume est peuplé une première fois via une tâche Fargate de « bootstrap » qui copie les fichiers depuis S3 vers l'EFS.

8.5 Identités et permissions (IAM)

Deux rôles IAM distincts interviennent sur la tâche ECS, chacun avec sa politique de confiance autorisant `ecs-tasks.amazonaws.com` à l'assumer :

- **`kojin-ecs-execution-role`** porte la politique managée `AmazonECSTaskExecutionRolePolicy` (pull ECR, écriture des logs CloudWatch) ;
- **`kojin-ecs-task-role`** est utilisé par le code applicatif : il porte une politique de lecture S3 restreinte au bucket de données et une politique d'invocation Bedrock.

Aucune clé d'accès statique n'est stockée dans le code ni dans l'image Docker : l'authentification applicative repose entièrement sur ce rôle de tâche.

```
{ "Version": "2012-10-17", "Statement": [{
  "Effect": "Allow",
  "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
  "Resource": [
    "arn:aws:bedrock:*:foundation-model/*",
    "arn:aws:bedrock:us-east-1:<compte>:inference-profile/*"
  ]
}] }
```

8.6 Pipeline de mise en production

La mise en production a suivi une séquence manuelle et reproductible, documentée dans `DEPLOYMENT.md` : construction de l'image Docker, publication vers ECR, création/mise à jour de la définition de tâche, puis mise à jour du service ECS qui bascule le trafic vers la nouvelle tâche. Deux mises à jour de service ont été nécessaires lors du déploiement réel pour corriger, respectivement, un problème d'architecture de conteneur et un problème de modèle Bedrock (§8.11) — chacune résolue par un simple `update-service --force-new-deployment` après correction de la task definition, sans interruption longue de service. Il n'y a pas, à ce stade, d'intégration continue automatisée (pas de dossier `.github/workflows`) : il s'agit d'une piste d'amélioration identifiée (§12.2).

8.7 Supervision, alertes et FinOps

Les journaux applicatifs sont collectés dans le groupe CloudWatch Logs /ecs/kojin. Un socle d'alarmes est recommandé (CPU > 80 % pendant 10 min, mémoire > 85 % pendant 5 min, taux d'erreur 5xx > 1 %, UnHealthyHostCount ≥ 1).

Sur le plan FinOps, un **budget AWS** (kojin-monthly-budget, plafond 30 \$/mois) a été mis en place avec deux alertes par e-mail : à 50 % de la dépense réelle et à 100 % de la dépense prévisionnelle, avant même le lancement du déploiement — une mesure de sobriété budgétaire directement motivée par le passage d'un compte AWS Academy (temporaire, budget préalloué) à un compte AWS personnel (facturation réelle et continue).

8.8 Estimation des coûts (FinOps)

Ressource	Configuration	Coût mensuel estimé (usage continu)
ECS Fargate	1 tâche, 2 vCPU / 8 Go ARM64, 24 h/24	≈ 60 €
Application Load Balancer	1 ALB, trafic faible	≈ 18 €
Amazon EFS	~1 Go (kojin.db + recipperl.pt)	≈ 0,3 €
Amazon ECR	~0,7 Go stocké (image optimisée)	< 1 €
CloudWatch Logs	≈ 1 Go	≈ 0,6 €
Amazon S3	< 1 Go (CSV + checkpoint de secours)	< 0,1 €
Total (usage continu)		≈ 80 €/mois

Ce coût mensuel suppose un fonctionnement continu ; pour l'usage réel du projet (déploiement de démonstration sur quelques jours), le coût facturé est proportionnellement bien inférieur, AWS facturant à l'heure d'usage effectif et non au forfait mensuel. Le dimensionnement 2 vCPU / 8 Go — supérieur à celui envisagé initialement (1 vCPU / 4 Go) — s'explique entièrement par le RL Planner ; sans cette page, le coût redescendrait proche de 50 €/mois.

8.9 Checklist de mise en production et nettoyage

Le guide de déploiement se conclut par une checklist de mise en production (bucket versionné et fermé au public, volume EFS peuplé, image sans CSV ni checkpoint embarqués, rôles IAM créés, task definition avec volume EFS et architecture ARM64 déclarée, ALB avec sticky sessions, security groups restreints, budget configuré) ainsi qu'une procédure de nettoyage complète (arrêt du service, suppression du cluster, du load balancer, de l'EFS, du dépôt ECR et du bucket S3), utile pour maîtriser les coûts entre deux démonstrations — le déploiement de ce projet étant, par construction, temporaire.

8.10 Variables d'environnement principales

Variable	Valeur en production	Rôle
LLM_PROVIDER	bedrock	Force l'usage de Bedrock (pas de repli auto en production)
BEDROCK_MODEL_ID	us.amazon.nova-pro-v1:0	Modèle Bedrock de référence (voir §8.11 pour la justification du choix)
AWS_REGION	us-east-1	Région AWS où le modèle est invoqué
DATA_S3_URI	s3://kojin-data-<compte>/products_names_with_macro_nutriments.csv	Emplacement S3 du CSV de secours
STREAMLIT_SERVER_HEADLESS	true	Démarrage sans ouverture de navigateur local

8.11 Incidents de déploiement rencontrés et résolus

Le déploiement réel a mis en évidence trois incidents non anticipés dans la documentation initiale, corrigés au fil de l'eau et intégrés à `DEPLOYMENT.md` pour un prochain déploiement :

- 1. Image Docker surdimensionnée (9,8 Go).** `requirements.txt` référence torch sans préciser d'index ; sur une image Linux, pip installe par défaut la roue CUDA complète (bibliothèques NVIDIA), inutile sur Fargate qui ne dispose pas de GPU. Installer torch depuis l'index CPU officiel (<https://download.pytorch.org/whl/cpu>) avant le reste des dépendances a ramené l'image à 3,05 Go, réduisant d'autant le temps de publication vers ECR.
- 2. Incompatibilité d'architecture de conteneur.** L'image construite sur un poste de développement Apple Silicon est nativement `linux/arm64` ; la tâche ECS, faute de déclaration explicite, attend par défaut `linux/amd64`, provoquant un échec `CannotPullContainerError`. La correction retenue déclare `runtimePlatform: {cpuArchitecture: ARM64}` dans la définition de tâche plutôt que de reconstruire l'image pour `x86_64` par émulation — solution à la fois plus rapide et moins coûteuse (Fargate Graviton).
- 3. Blocage administratif sur le modèle Bedrock Anthropic.** Le modèle initialement retenu pour sa qualité de génération SQL (`us.anthropic.claude-sonnet-4-6`) a échoué en production avec l'erreur `ResourceNotFoundException: Model use case details have not been submitted for this account` — une exigence de conformité propre aux modèles tiers (Anthropic) sur Bedrock, nécessitant de remplir un formulaire d'usage prévu et d'attendre une validation. Pour ne pas bloquer la démonstration, l'équipe a basculé sur **Amazon Nova Pro** (`us.amazon.nova-pro-v1:0`), un modèle propriétaire AWS non soumis à cette exigence tierce, immédiatement opérationnel après un simple redéploiement de la task definition.

9. Organisation du projet et travail en équipe

9.1 Équipe et référent fonctionnel

Membre	Classe
MOREL Tom	MAS_SE2_000_ALT – 5ESGI IABD CL A ALT RO
EL KHEIR Douaa	MAS_SE2_000_ALT – 5ESGI IABD CL A ALT RO
ELKADOURI Hajar	MAS_SE2_000_ALT – 5ESGI IABD CL A ALT RO
ZAIDI Yassine	MAS_SE2_000_ALT – 5ESGI IABD CL A ALT RO

Référent pédagogique : Mme Amrita Devy BALASOUPRAMANIANE.

9.2 Méthode de travail

L'équipe a fonctionné selon un rythme itératif calé sur les séances de suivi imposées par le référent pédagogique. Le code a été versionné sur un dépôt Git partagé (GitHub), avec relectures croisées avant fusion. La coordination quotidienne s'est faite via un serveur Discord dédié, en complément des échanges de code sur GitHub.

9.3 Répartition des tâches

Domaine	Membre référent	Charge estimée
Intégration LLM (Bedrock) et déploiement AWS	MOREL Tom	≈ 115 h
Interface Streamlit et expérience utilisateur	EL KHEIR Douaa	≈ 100 h
Moteur de calcul (nutrition + solveur NNLS/BVLS) et RL Planner	ELKADOURI Hajar	≈ 110 h
Pipeline de données (Open Food Facts, filtrage, tags)	ZAIDI Yassine	≈ 105 h

9.4 Planning et suivi des séances

Le projet a été suivi par le référent pédagogique au travers de trois séances de suivi avant la soutenance finale, chacune donnant lieu à un document de suivi partagé.

10. Qualité d'usage, tests et validation

L'installation et le lancement de l'application ont été testés en local (environnement virtuel Python 3.12, installation des dépendances, préparation des données, entraînement d'un checkpoint RL

Planner sur le dataset Food.com) ainsi qu'en configuration cloud réelle sur AWS, jusqu'à l'obtention d'une URL publique fonctionnelle.

10.1 Plan de tests fonctionnels

Cas de test	Résultat attendu	Statut
Saisie d'un profil complet	Objectifs journaliers affichés et cohérents avec Mifflin–St Jeor	Validé
Sélection d'un régime restrictif (ex. vegan)	Aucun produit hors régime dans les bentos générés	Validé
Répartition sur plusieurs bentos avec fractions couplées	La somme des fractions reste égale à 100 % après ajustement	Validé
Composition avec protéine animale désignée sur un seul bento	Aucune autre occurrence de protéine animale sur les autres bentos	Validé
Entraînement et chargement du checkpoint RL Planner	Génération d'une journée complète de recettes sans erreur	Validé
Requête en langage naturel (page Exploration)	SQL généré et exécuté correctement sur DuckDB	Validé (après bascule sur Nova Pro, §8.11)
Premier lancement sans CSV local	Le bouton de préparation des données apparaît et déclenche le pipeline	Validé
Déploiement AWS de bout en bout	Application accessible publiquement, health check HTTP 200	Validé
Accès depuis un poste tiers (hors machine de déploiement)	Interface identique accessible via l'URL publique	Validé

11. Gestion des risques

Risque	Impact	Mitigation retenue
Indisponibilité ou restriction d'usage d'un modèle Bedrock tiers (Anthropic)	Fonctionnalité LLM indisponible en production	Bascule vers un modèle propriétaire AWS (Nova Pro) non soumis à validation administrative tierce (incident réel, §8.11)
Incompatibilité d'architecture de conteneur (ARM64 vs amd64)	Échec de démarrage de la tâche ECS	Déclaration explicite de runtimePlatform dans la task definition (incident réel, §8.11)
Durée de préparation des données trop longue en démonstration	Blocage lors d'une démonstration live	Préparation du CSV en amont et dépôt sur S3 avant toute présentation

Risque	Impact	Mitigation retenue
Qualité hétérogène des fiches Open Food Facts	Compositions de bento peu réalistes	Filtrage sur produits non obsolètes, exclusion NOVA 4, tags de régime vérifiés
Perte de données utilisateur au redémarrage d'une tâche Fargate sans état	Profils et checkpoint RL perdus à chaque redéploiement	Volume EFS persistant monté sur /app/data
Dérive de coût cloud (passage à un compte personnel facturé en continu)	Dépassement budgétaire non détecté	Budget AWS avec alertes automatiques à 50 %/100 %, nettoyage complet des ressources prévu en fin de démonstration
Dépendance à un seul membre sur un domaine technique	Blocage en cas d'indisponibilité	Relecture croisée systématique du code avant fusion

12. Bilan de réalisation

12.1 Limites actuelles

- La qualité nutritionnelle de certains produits Open Food Facts reste hétérogène malgré le filtrage appliqué
- La préparation initiale des données reste une étape longue (plusieurs dizaines de minutes selon la stabilité de la connexion réseau) qui doit être anticipée avant une démonstration
- Le solveur repose sur des bornes de portion par défaut (200 g) non encore personnalisables par l'utilisateur
- Le RL Planner souffre d'un problème de cold-start pour tout nouvel utilisateur (non présent dans les 2 855 profils Food.com d'entraînement) : la personnalisation ne s'opère que progressivement, par accumulation de journées validées
- Le pipeline de mise en production n'est, à ce stade, pas automatisé de bout en bout (pas de CI/CD)
- Le déploiement AWS réalisé est temporaire par nature (compte personnel, nettoyage prévu en fin de démonstration) et non conçu comme un service permanent

12.2 Améliorations possibles

- Mettre en place une intégration continue déclenchant automatiquement le déploiement ECS depuis la branche principale
- Ajouter des alarmes CloudWatch actives (taux d'erreur, latence) pour un suivi proactif
- Ajouter un certificat TLS (ACM) et un nom de domaine pour un accès HTTPS
- Réentraîner le RL Planner sur les interactions réelles des utilisateurs Kōjin plutôt que sur le seul simulateur Food.com
- Soumettre le formulaire d'usage Anthropic pour réévaluer l'usage de Claude en complément de Nova Pro

12.3 Perspectives à court, moyen et long terme

Horizon	Objectif	Statut
Court terme	Finaliser la documentation, déploiement AWS fonctionnel, maintenance	Réalisé
Moyen terme	Mettre en place l'intégration continue et les alarmes de supervision	Non démarré
Long terme	Historique utilisateur enrichi, filtrage qualité avancé, mode multi-jours	Piste identifiée

13. Glossaire

Terme	Définition
BMR	Métabolisme de base (Basal Metabolic Rate)
TDEE	Dépense énergétique totale journalière
NNLS	Non-Negative Least Squares
BVLS	Bounded-Variable Least Squares
NCF	Neural Collaborative Filtering — modélisation des préférences utilisateur/item par réseau de neurones
PPO	Proximal Policy Optimization — algorithme d'apprentissage par renforcement
NOVA	Classification des aliments selon leur degré de transformation
ODbL	Open Database License
ECS Fargate	Service AWS de conteneurs serverless
Graviton	Famille de processeurs ARM64 conçus par AWS, alternative moins coûteuse à x86 sur Fargate
EFS	Elastic File System — système de fichiers réseau persistant AWS
ALB	Application Load Balancer
IAM	Identity and Access Management
Bedrock	Service AWS d'accès à des modèles de langage managés
FinOps	Pratiques de pilotage et d'optimisation des coûts liés au cloud

14. Annexes

14.1 Structure du dépôt

```
kojin/
├─ README.md           – documentation générale du projet
├─ DEPLOYMENT.md       – déploiement AWS (ECS Fargate ARM64 + S3 + EFS + ALB)
├─ requirements.txt     – dépendances Python
├─ Dockerfile          – image applicative (torch CPU-only, reciperl/ inclus)
├─ streamlit_app.py     – point d'entrée, navigation
├─ kojिन_common.py     – CSS, constantes, solveur, factory LLM
├─ app_pages/          – Profil, Bento Planner, Exploration, RL Planner
├─ reciperl/           – NCF, PPO, environnement RL, chargement Food.com
├─ data_prep_nutriments.py – pipeline Open Food Facts → CSV
└─ data/               – non versionné : food.parquet, CSV préparé, kojिन.db,
reciperl.pt
```

14.2 Sources et crédits

- Open Food Facts — base de données produits, licence ODbL
- Food.com Recipes and Reviews (Kaggle) — entraînement du RL Planner
- Hugging Face Hub — hébergement du dataset openfoodfacts/product-database
- Amazon Web Services — Bedrock, ECS Fargate, EFS, S3, CloudWatch, IAM
- Liu et al., “An Interactive Food Recommendation System Using Reinforcement Learning”, 2024 — architecture du RL Planner

14.3 Accès à l'application déployée

<http://kojin-alb-1461380392.us-east-1.elb.amazonaws.com>

Accessible publiquement depuis n'importe quel navigateur, sans installation ni configuration préalable.

15. Conclusion

Le projet Kōjin a permis à l'équipe de mettre en œuvre, sur un cas d'usage concret, l'ensemble de la chaîne data-produit vue durant l'année : ingestion et nettoyage d'un jeu de données volumineux, modélisation mathématique d'un problème d'optimisation sous contraintes, apprentissage par renforcement sur des données réelles, intégration d'un modèle de langage cloud, et déploiement d'une application conteneurisée et sécurisée sur AWS jusqu'à son exposition publique. Les objectifs fixés en début de projet — un outil qui compose automatiquement des repas équilibrés, avec une alternative de planification interactive par renforcement — ont été atteints : l'application est aujourd'hui déployée et accessible à l'adresse **<http://kojin-alb-1461380392.us-east-1.elb.amazonaws.com>**, et l'ensemble des cas de test fonctionnels a été validé (§10.1).

Le déploiement réel a également été l'occasion de résoudre trois incidents techniques non anticipés (image Docker surdimensionnée, incompatibilité d'architecture ARM64/amd64, restriction administrative sur un modèle Bedrock tiers), documentés en §8.11 : au-delà de la note, cette capacité à diagnostiquer et corriger des problèmes d'infrastructure réels en conditions de déploiement constitue une compétence directement transférable en environnement professionnel.